

ASSIGNMENT-15.3

Name: G.Mythili

Ht.no:2303A51283

Batch:05

Task 1:Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

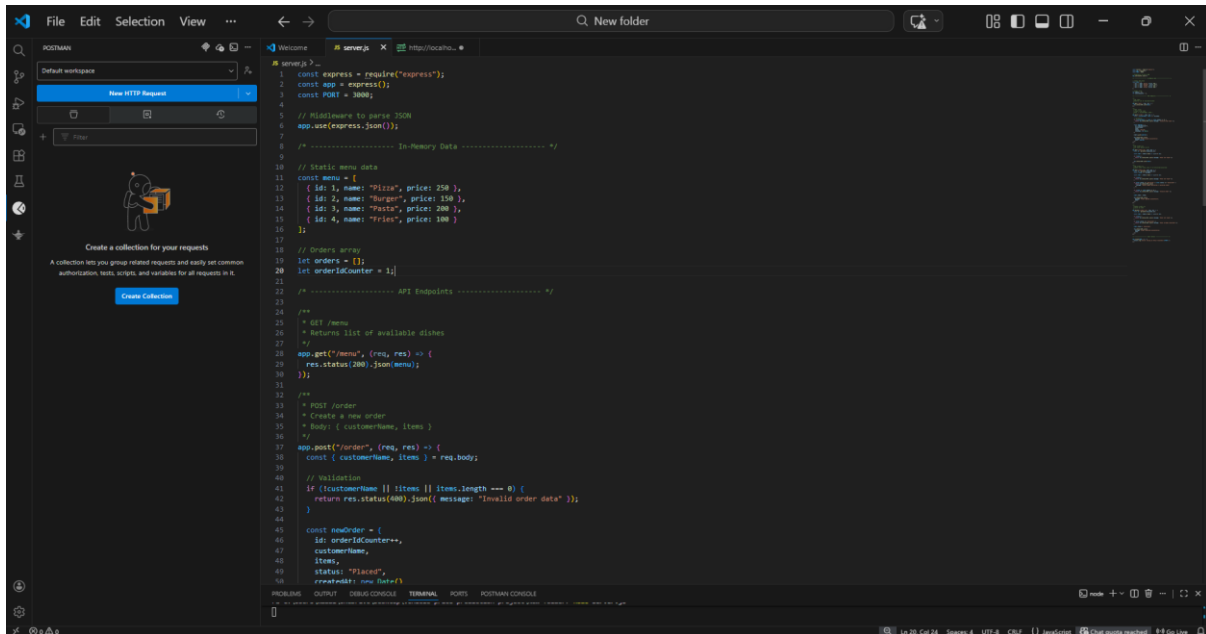
- Endpoints required:
- GET /menu → List available dishes
- POST /order → Place a new order
- GET /order/{id} → Track order status
- PUT /order/{id} → Update an existing order (e.g., change items)
- DELETE /order/{id} → Cancel an order
- AI should generate:
- REST API code
- Suggested improvements (like authentication, pagination)

Prompt:

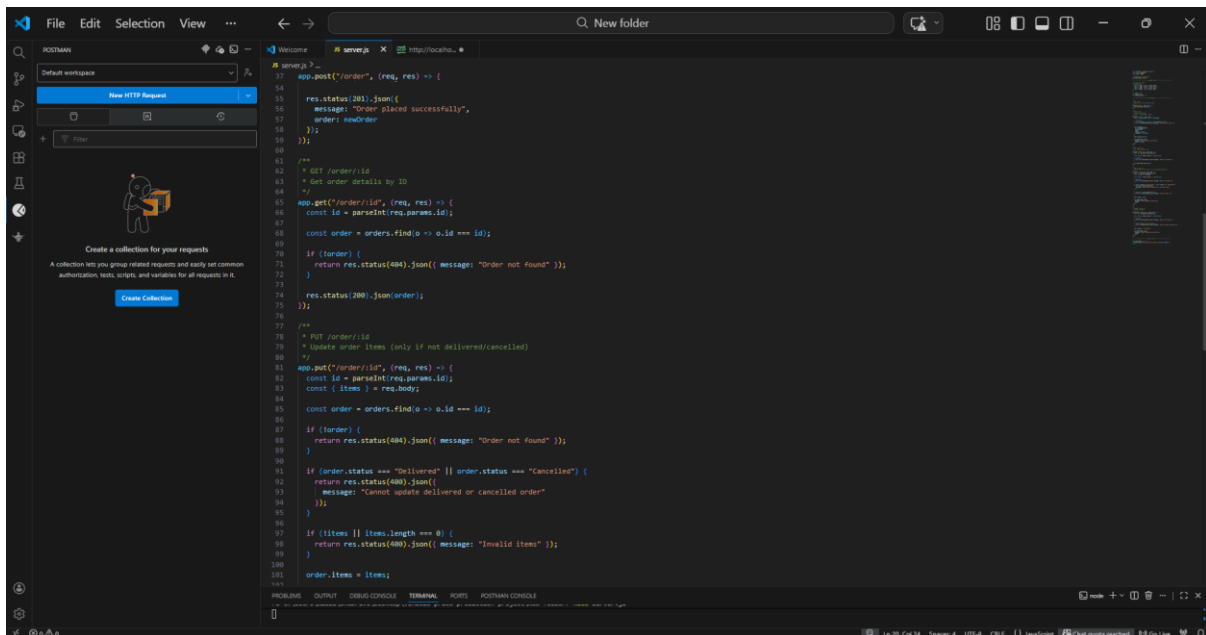
```
Create a Node.js Express REST API for an online food ordering system.  
Requirements: - GET /menu → return list of dishes - POST /order → create new order with items - GET /order/:id → get order status - PUT /order/:id → update order items - DELETE /order/:id → cancel order  
Use in-memory arrays. Add proper comments for each endpoint.
```

Code:

server.js

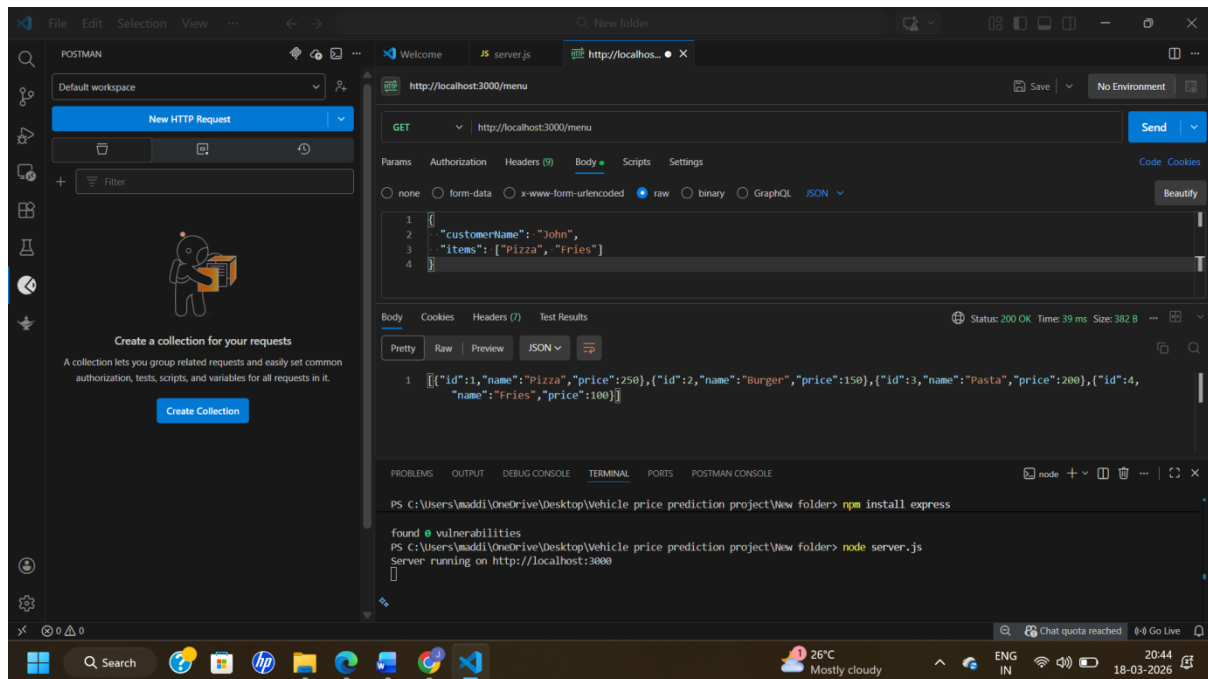


```
1 const express = require("express");
2 const app = express();
3 const PORT = 3000;
4
5 // Middleware to parse JSON
6 app.use(express.json());
7
8 /* ----- In-Memory Data ----- */
9
10 // Static menu data
11 const menu = [
12   { id: 1, name: "Pizza", price: 250 },
13   { id: 2, name: "Burger", price: 150 },
14   { id: 3, name: "Pasta", price: 200 },
15   { id: 4, name: "Fries", price: 100 }
16 ];
17
18 // Orders array
19 let orders = [];
20 let orderCounter = 1;
21
22 /* ----- API Endpoints ----- */
23
24 // GET /menu
25 // Returns list of available dishes
26 app.get("/menu", (req, res) => {
27   res.status(200).json(menu);
28 });
29
30 // POST /order
31 // Create a new order
32 // Body: { customerName, items }
33 app.post("/order", (req, res) => {
34   const { customerName, items } = req.body;
35
36   // Validation
37   if (!customerName || items.length === 0) {
38     return res.status(400).json({ message: "Invalid order data" });
39   }
40
41   const newOrder = {
42     id: orderCounter++,
43     customerName,
44     items,
45     status: "placed",
46     createdAt: new Date()
47   };
48   orders.push(newOrder);
49   res.status(201).json(newOrder);
50 });
```

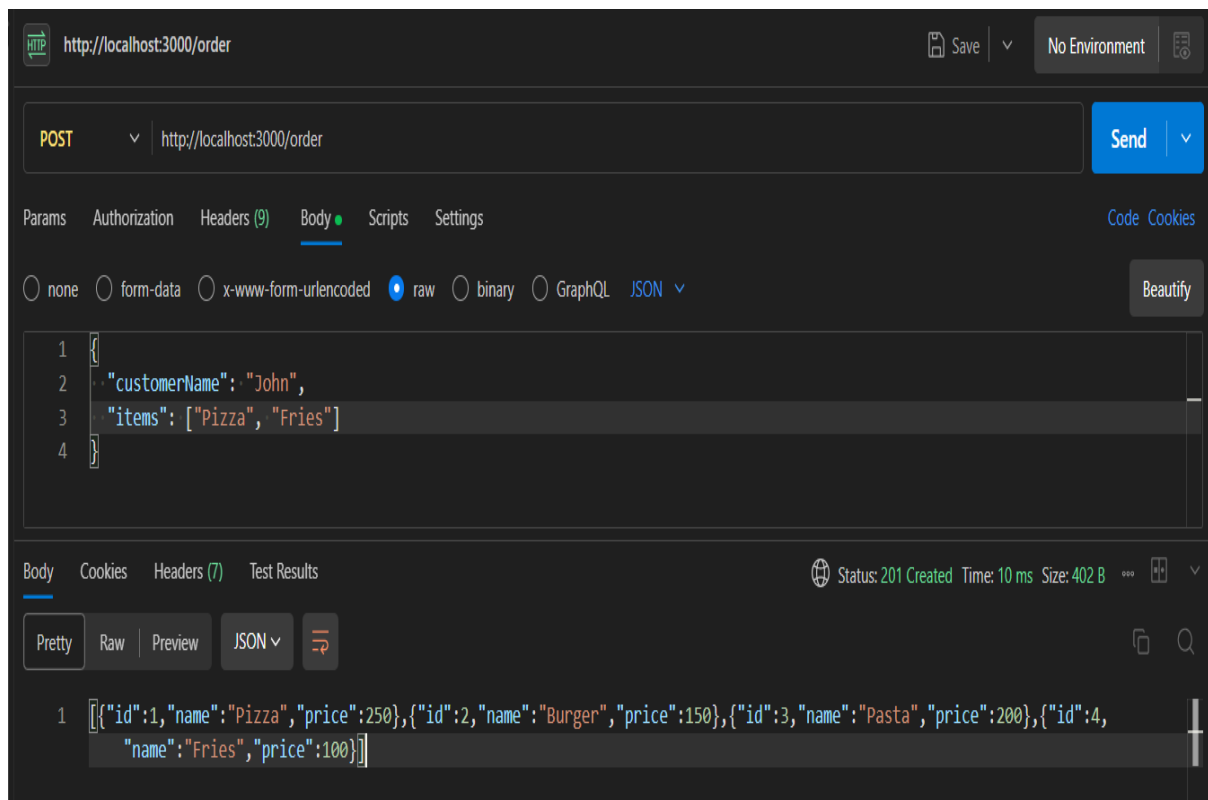


```
17 app.post("/order", (req, res) => {
18   res.status(201).json({
19     message: "Order placed successfully",
20     order: newOrder
21   });
22 });
23
24 // GET /order/:id
25 // Get order details by ID
26 app.get("/order/:id", (req, res) => {
27   const id = parseInt(req.params.id);
28   const order = orders.find(o => o.id === id);
29
30   if (!order) {
31     return res.status(404).json({ message: "Order not found" });
32   }
33   res.status(200).json(order);
34 });
35
36 // PUT /order/:id
37 // Update order items (only if not delivered/cancelled)
38 app.put("/order/:id", (req, res) => {
39   const id = parseInt(req.params.id);
40   const { items } = req.body;
41   const order = orders.find(o => o.id === id);
42
43   if (!order) {
44     return res.status(404).json({ message: "Order not found" });
45   }
46
47   if (order.status === "delivered" || order.status === "cancelled") {
48     return res.status(400).json({
49       message: "Cannot update delivered or cancelled order"
50     });
51   }
52
53   if (!items || items.length === 0) {
54     return res.status(400).json({ message: "Invalid items" });
55   }
56
57   order.items = items;
58   res.status(200).json(order);
59 });
```

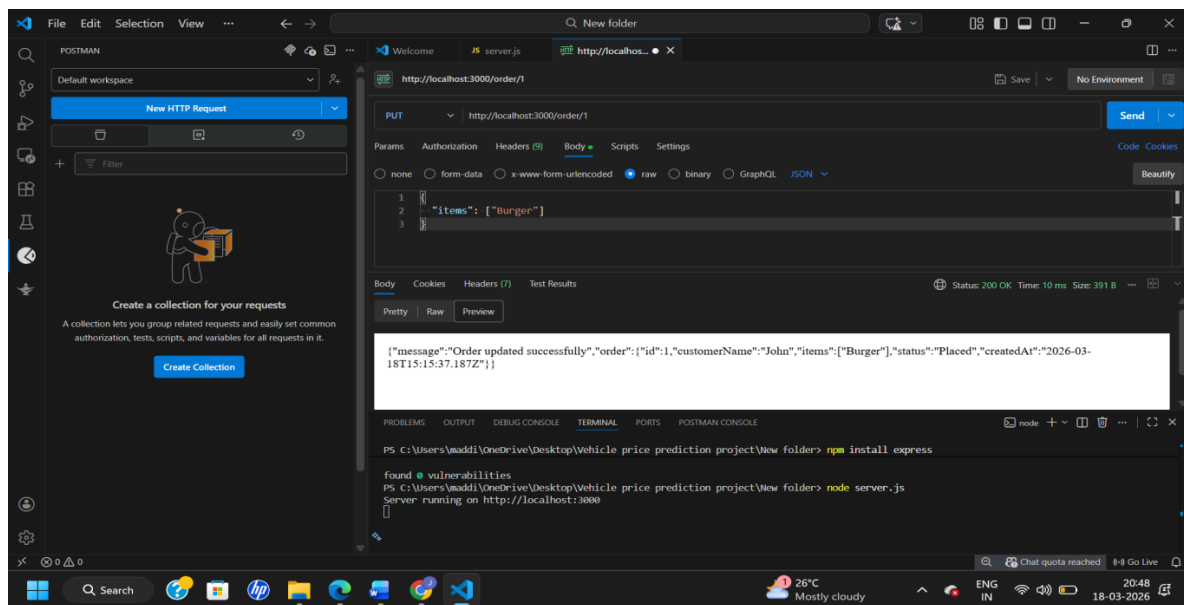
Get:



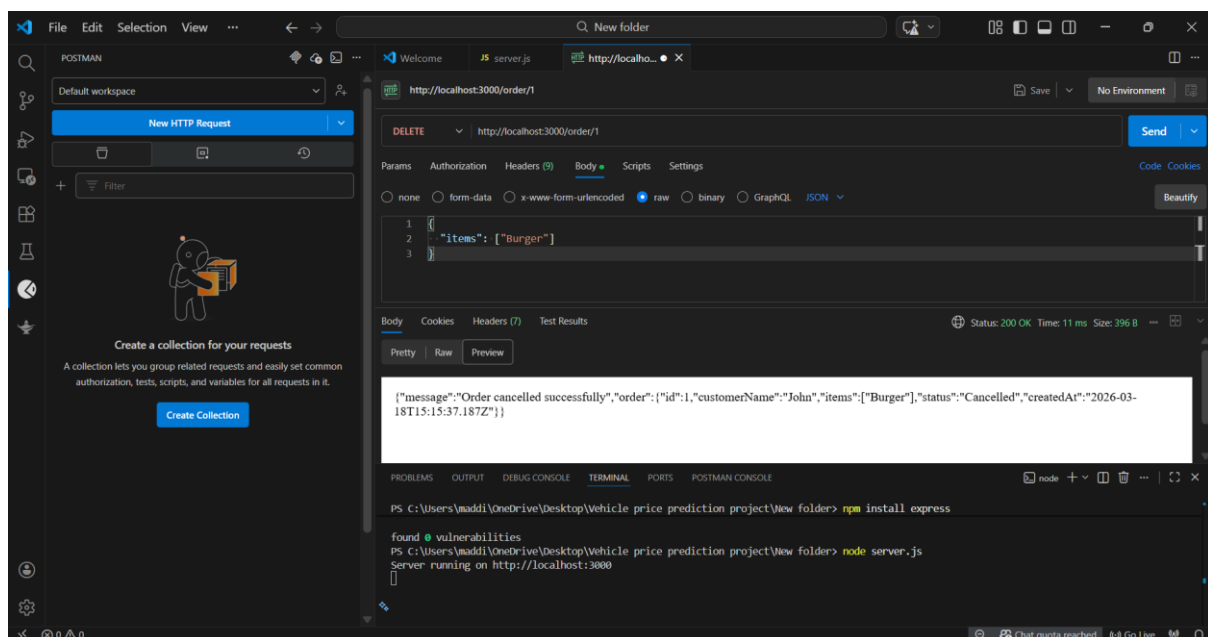
Post:



Put:



Delete:



Explanation:

This API simulates a basic online food ordering system using REST principles.

The GET `/menu` endpoint retrieves available dishes.

The POST `/order` endpoint allows users to place a new order by sending selected items.

The **GET /order/:id** endpoint helps in tracking the status of an order. The **PUT /order/:id** endpoint is used to update order details such as items.

The **DELETE /order/:id** endpoint cancels an existing order.

All data is stored in an in-memory array, making it simple and fast for testing.

Task 2:Employee Payroll API

Scenario:

Create an API for managing employees and their salaries.

Instructions:

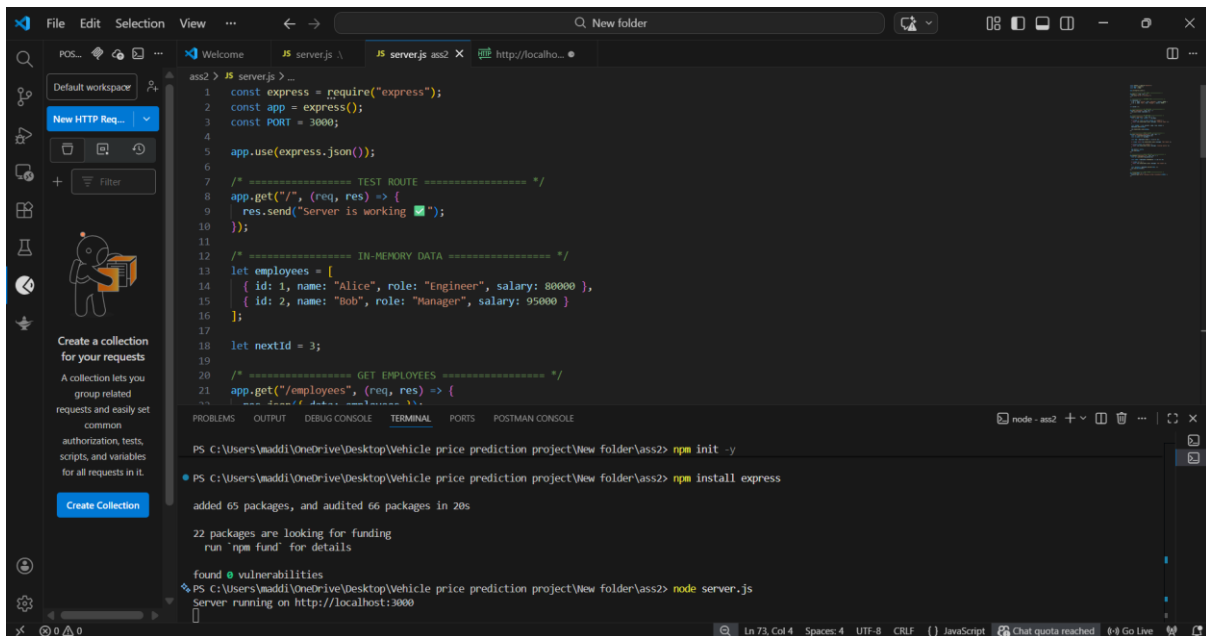
- Endpoints required:
- GET /employees → List all employees
- POST /employees → Add a new employee with salary details
- PUT /employees/{id}/salary → Update salary of an employee
- DELETE /employees/{id} → Remove employee from system
- Use AI to:
- Suggest data model structure.
- Add comments/docstrings for all endpoints.

Prompt:

Create a Node.js Express REST API to manage employees and salaries.

Requirements: - GET /employees → list all employees
- POST /employees → add employee with name, role, salary - PUT /employees/:id/salary → update salary - DELETE /employees/:id → remove employee
Add validation and comments for each endpoint.
Also suggest a proper data model.

Code:



```
1 const express = require("express");
2 const app = express();
3 const PORT = 3000;
4
5 app.use(express.json());
6
7 /* ===== TEST ROUTE ===== */
8 app.get("/", (req, res) => {
9   res.send("Server is working 🟢");
10 });
11
12 /* ===== IN-MEMORY DATA ===== */
13 let employees = [
14   { id: 1, name: "Alice", role: "Engineer", salary: 80000 },
15   { id: 2, name: "Bob", role: "Manager", salary: 95000 }
16 ];
17
18 let nextId = 3;
19
20 /* ===== GET EMPLOYEES ===== */
21 app.get("/employees", (req, res) => {
22   // TODO: fetch data from database
23 })
```

PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm init -y

PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm install express

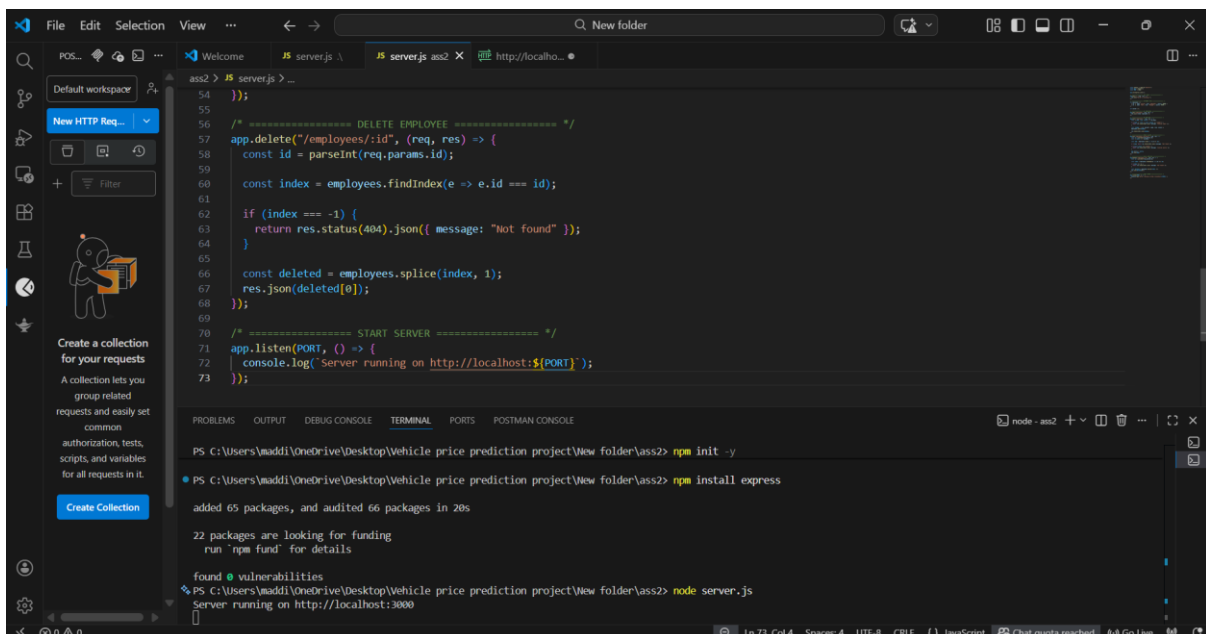
added 65 packages, and audited 66 packages in 20s

22 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> node server.js

Server running on http://localhost:3000



```
54 });
55
56 /* ===== DELETE EMPLOYEE ===== */
57 app.delete("/employees/:id", (req, res) => {
58   const id = parseInt(req.params.id);
59
60   const index = employees.findIndex(e => e.id === id);
61
62   if (index === -1) {
63     return res.status(404).json({ message: "Not found" });
64   }
65
66   const deleted = employees.splice(index, 1);
67   res.json(deleted[0]);
68 });
69
70 /* ===== START SERVER ===== */
71 app.listen(PORT, () => {
72   console.log(`Server running on http://localhost:${PORT}`);
73 });
```

PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm init -y

PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm install express

added 65 packages, and audited 66 packages in 20s

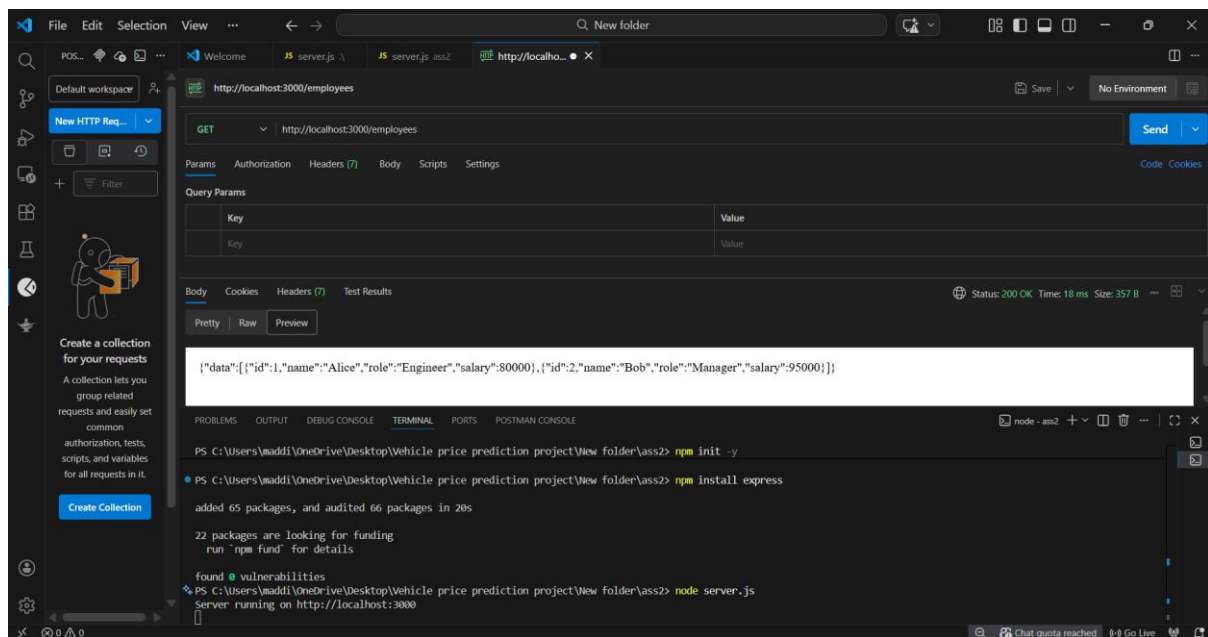
22 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

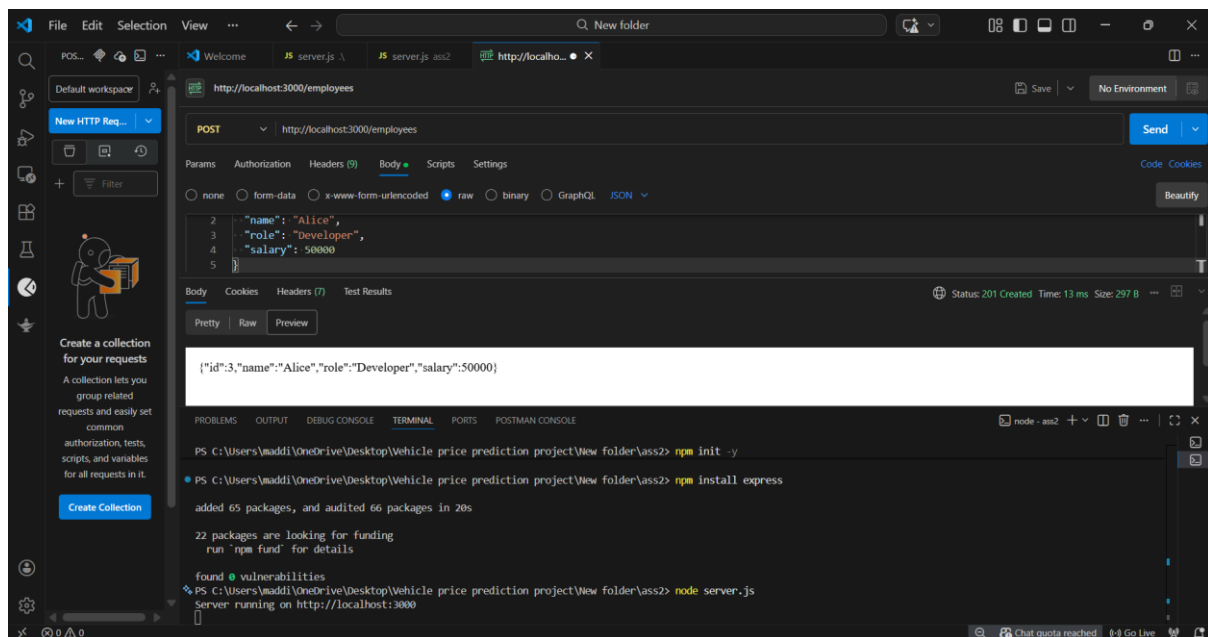
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> node server.js

Server running on http://localhost:3000

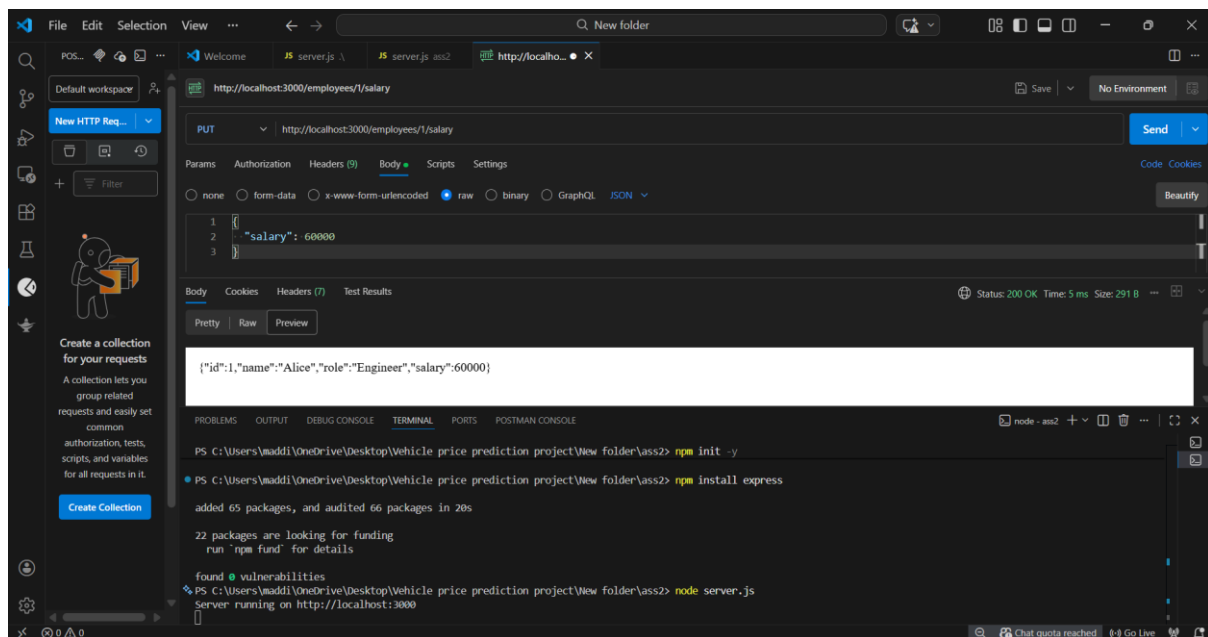
Get:



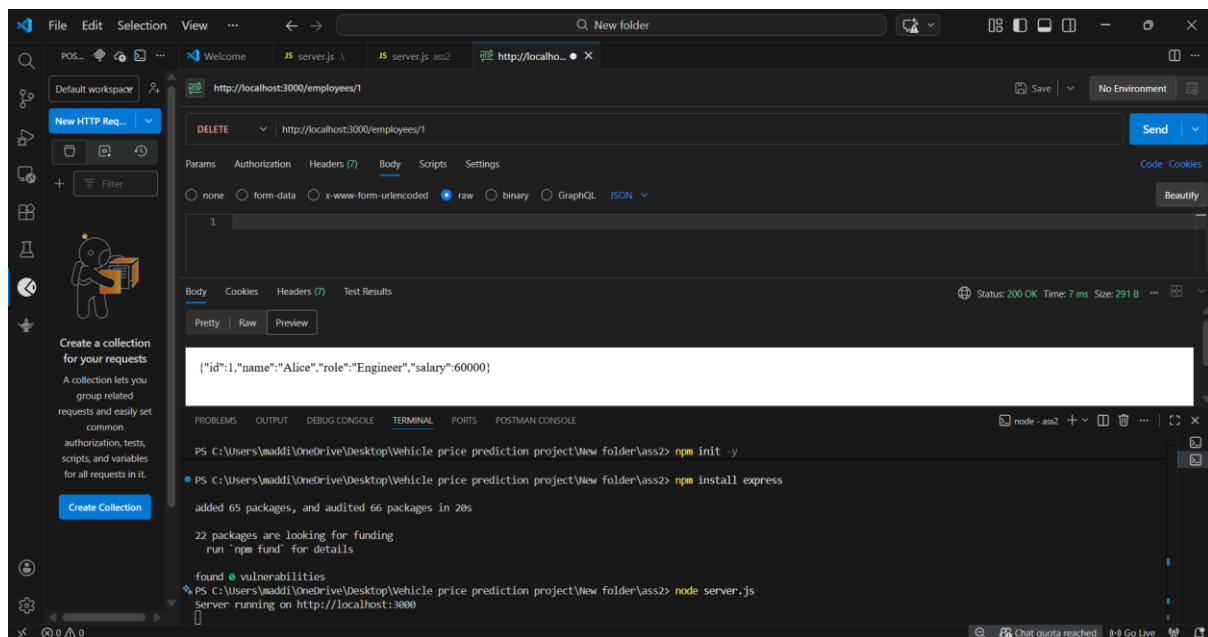
Post:



Put:



Delete:



Explanation:

his API is used to manage employee details and their salaries.

The **GET /employees** endpoint shows all employees.

The **POST /employees** endpoint adds a new employee with name, role, and salary.

The **PUT /employees/:id/salary** endpoint updates the salary of a specific employee.

The **DELETE /employees/:id** endpoint removes an employee from the system.

Basic validation is used to make sure correct data is entered before processing.

Task 3:Student Information REST API

Scenario:

A university wants to digitize student records for easy retrieval. You need to build

a Student Management API that allows:

- Adding a new student (name, roll number, course, year)
- Fetching details of all students
- Updating student information
- Deleting a student record

Implementation Instructions:

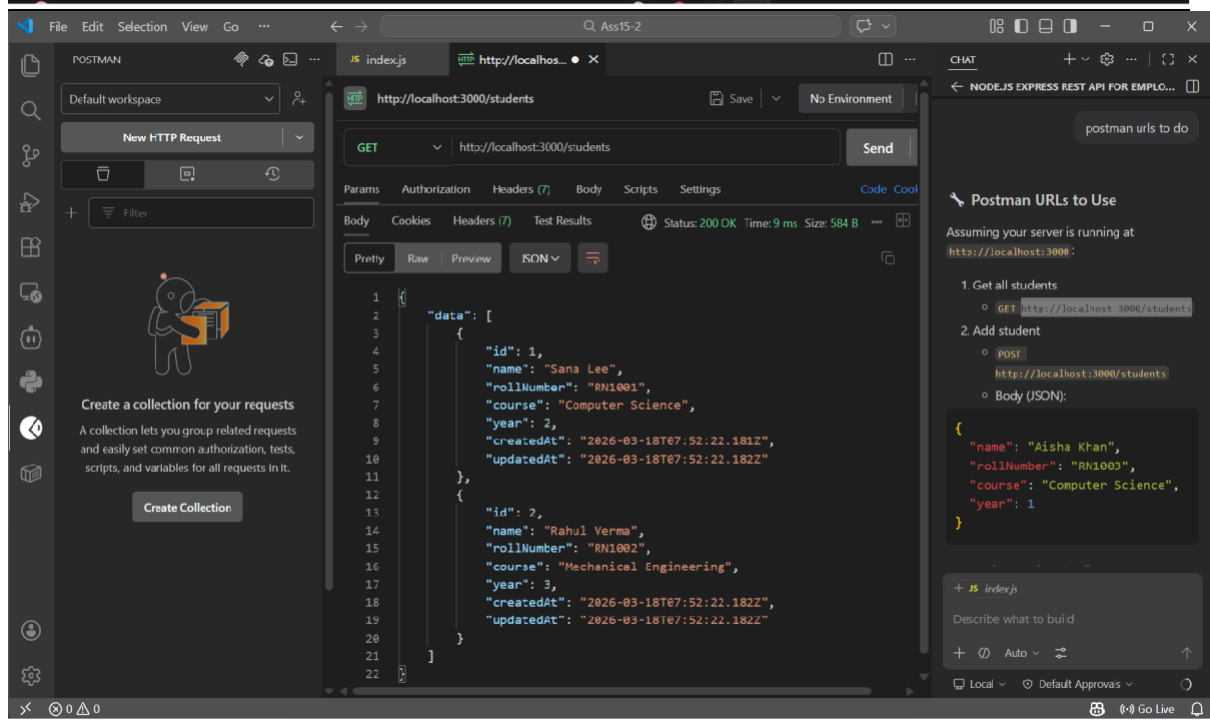
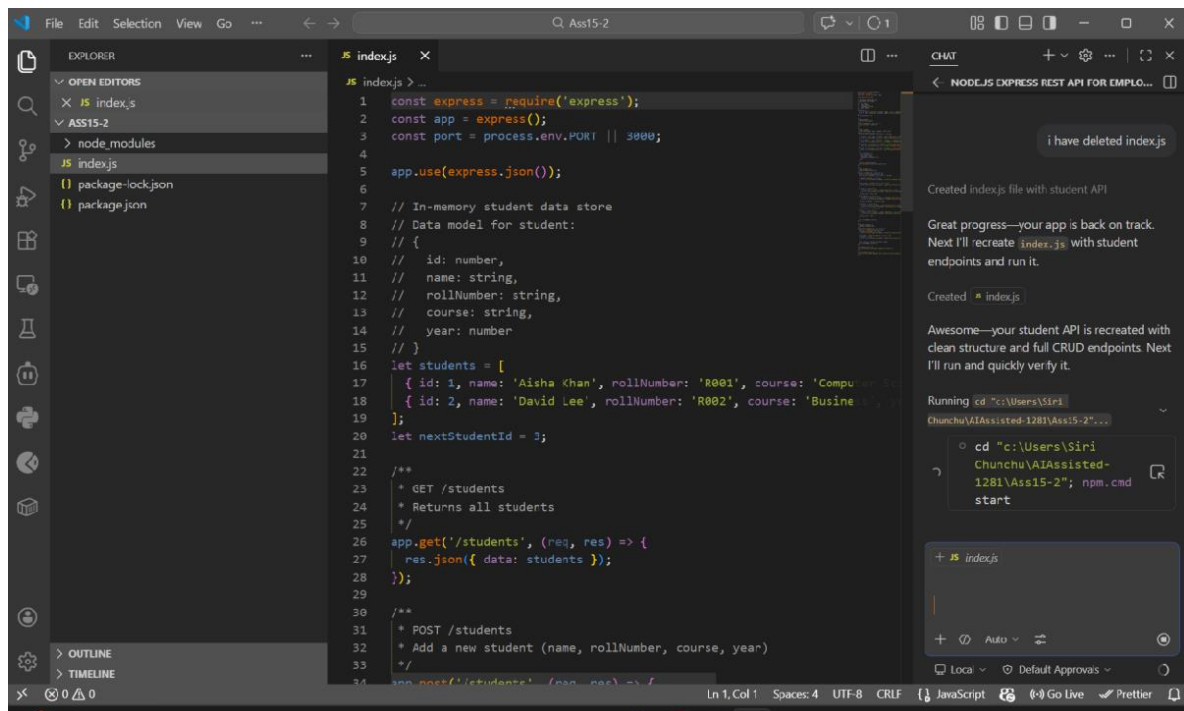
1. Use Node.js + Express (or Python Flask/Django REST) with GitHub Copilot to auto-generate routes.
2. Store student data in an in-memory array or SQLite/PostgreSQL database.

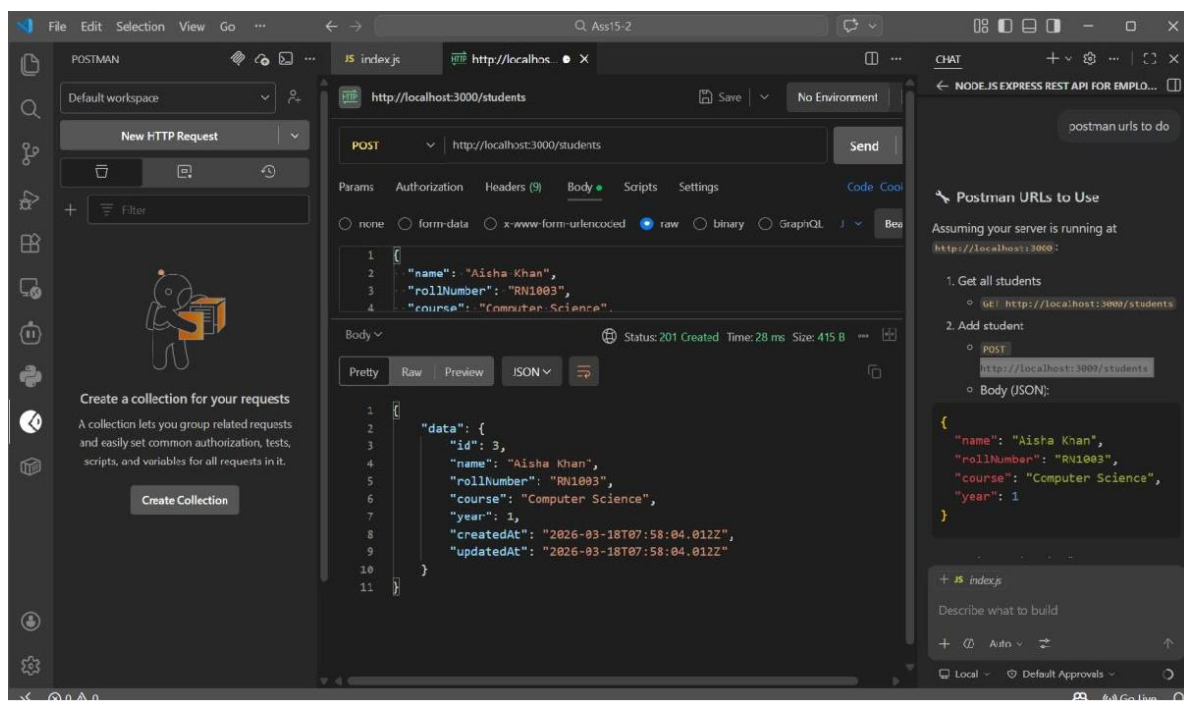
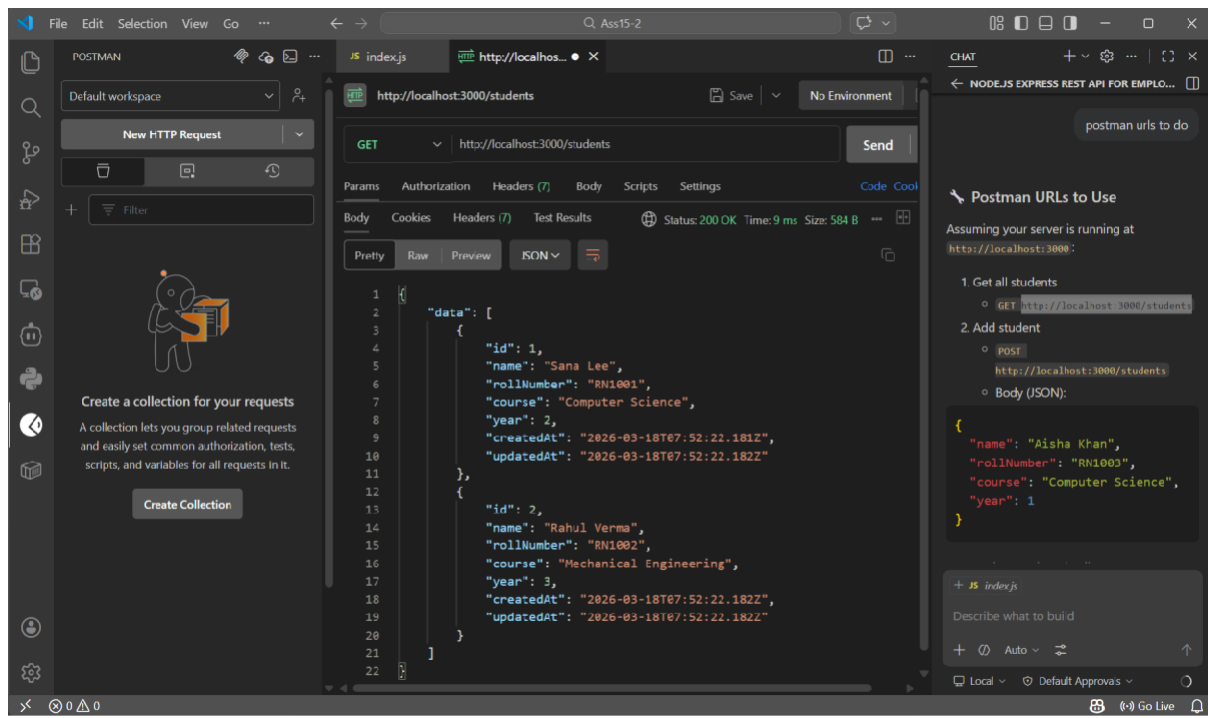
Test API endpoints with Postman.

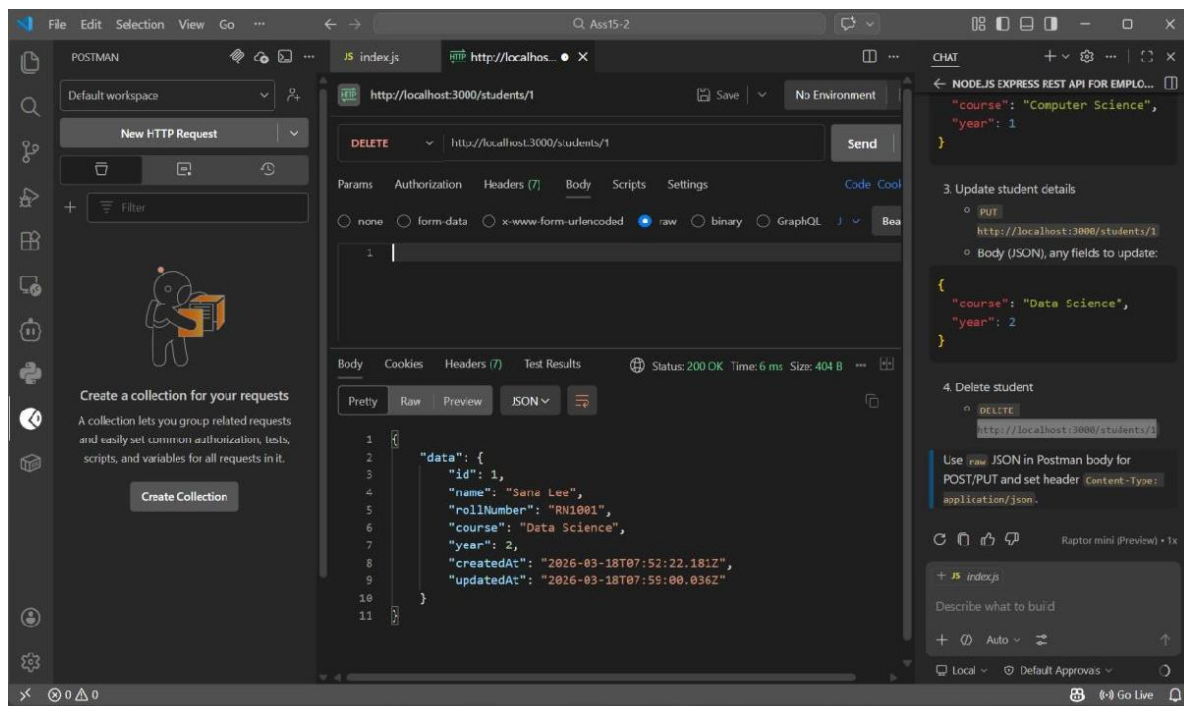
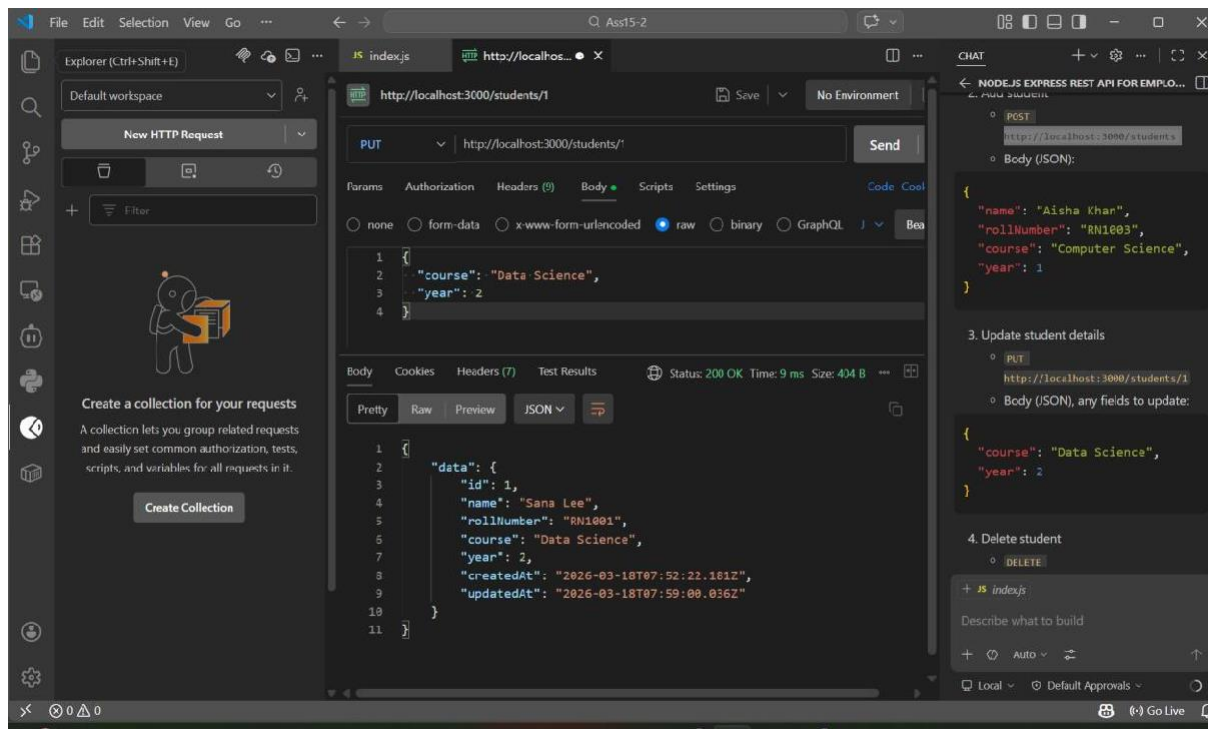
Prompt:

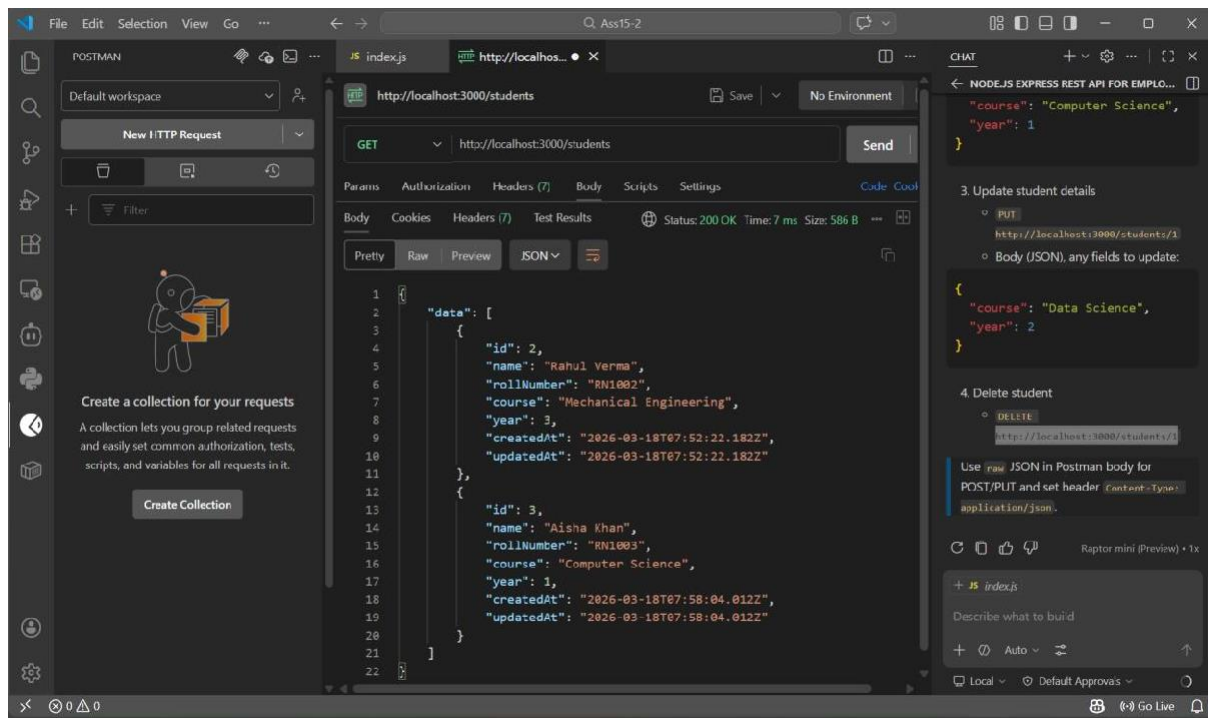
```
Create a Student Management REST API using
Node.js and Express.
Features: - Add student (name, roll number, course,
year) - Get all students - Update student details -
Delete student
Store data in an in-memory array.
Add comments and clean structure.
```

Code:









Explanation:

This API is designed to manage student records in a university system.

The **POST /students** endpoint adds new student details.

The **GET /students** endpoint retrieves all student records.

The **PUT /students/:id** endpoint updates existing student information. The **DELETE /students/:id** endpoint deletes a student record.

Data is stored in an in-memory array for simplicity.

Task 4: Student Information REST API

Scenario:

A university wants to digitize student records for easy retrieval. You need to build

a Student Management API that allows:

- Adding a new student (name, roll number, course, year)
- Fetching details of all students
- Updating student information

- Deleting a student record

Implementation Instructions:

1. Use Node.js + Express (or Python Flask/Django REST) with GitHub Copilot to auto-generate routes.
2. Store student data in an in-memory array or SQLite/PostgreSQL database.

Test API endpoints with Postman.

Prompt:

Create a Node.js Express API that integrates with OpenWeatherMap API.

Endpoints:

- /weather/current/:city → return temperature, humidity, condition
- /weather/forecast/:city → return 5-day forecast Use axios for API calls and return structured JSON. Include error handling and comments.

Code:

```
1 const express = require('express');
2 const axios = require('axios');
3 const app = express();
4 const port = process.env.PORT || 3000;
5
6 // Use your OpenWeatherMap API key here or set OPENWEATHER_API_KEY
7 const OPENWEATHER_API_KEY = process.env.OPENWEATHER_API_KEY || 'YOUR_API_KEY_HERE';
8
9 if (!OPENWEATHER_API_KEY || OPENWEATHER_API_KEY === 'YOUR_API_KEY_HERE') {
10   console.warn('\n⚠ Warning: OpenWeatherMap API key is missing. Set up your key in .env or as an env var.\n');
11 }
12
13 /**
14  * Helper: Map OpenWeatherMap response to structured weather data
15  */
16 function mapCurrentWeather(data) {
17   return {
18     city: data.name,
19     country: data.sys?.country || null,
20     temperature: data.main?.temp,
21     humidity: data.main?.humidity,
22     condition: data.weather[0].description,
23     windSpeed: data.wind?.speed,
24     timestamp: data.dt ? new Date(data.dt * 1000).toISOString() : null;
25   };
26 }
27
28 /**
29  * GET /weather/current/:city
30  * Return current weather for a city
31  */
32 app.get('/weather/current/:city', async (req, res) => {
33   const city = req.params.city;
34   if (!city || city.trim().length === 0) {
35     res.status(400).json({ error: 'City name is required' });
36     return;
37   }
38   try {
39     const response = await axios.get(`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${OPENWEATHER_API_KEY}&units=metric`);
40     const weatherData = mapCurrentWeather(response.data);
41     res.json(weatherData);
42   } catch (error) {
43     console.error('Error fetching weather data:', error);
44     res.status(500).json({ error: 'Internal server error' });
45   }
46 });
47
48 // Start the server
49 app.listen(port, () => {
50   console.log(`Server is running on port ${port}`);
51 });
```

Next step

1. Stop the running Node process or use a different port.
2. Start again with your real API key:

```
set OPENWEATHER_API_KEY=your_real_key
set PORT=3001
npm start
```

Then test in Postman:

- GET
http://localhost:3001/weather/current/London
- GET
http://localhost:3001/weather/forecast/London

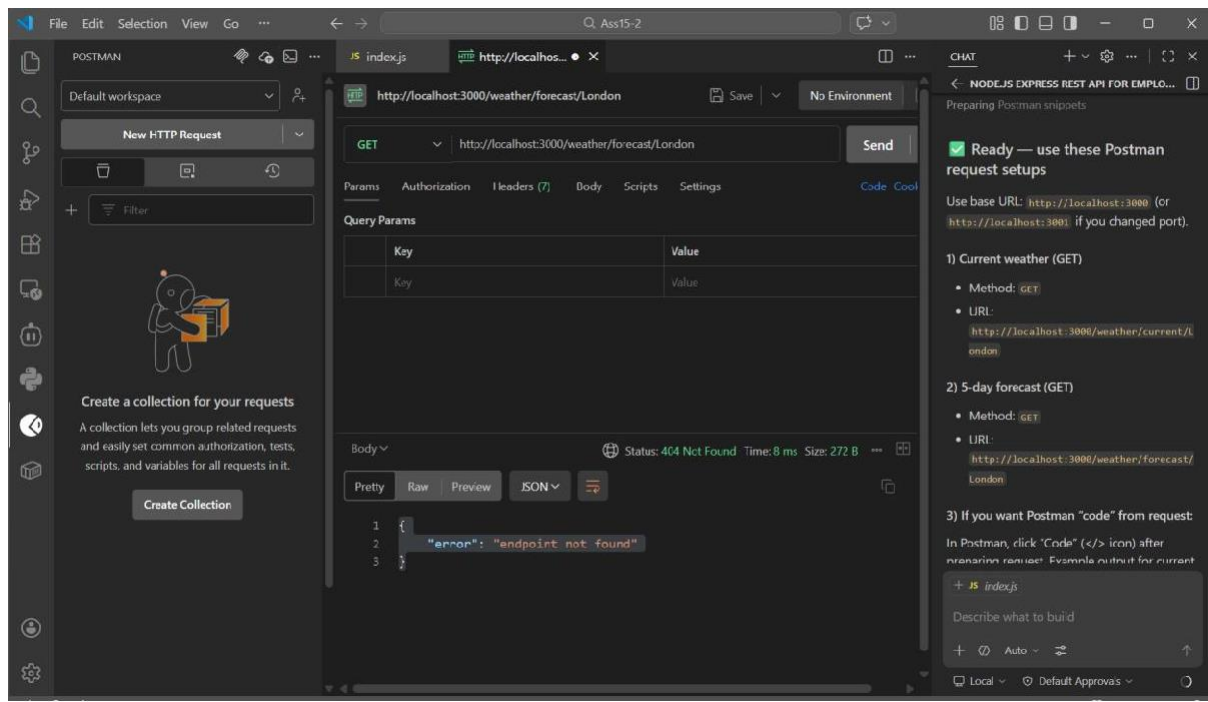
If you want, I can now add a small .env + dotenv setup so this is easier to run next time.

+ JS index.js

Describe what to build

+ Auto v

Local v Default Approvals v



Explanation:

This API integrates with an external service (OpenWeatherMap) to fetch real-time weather data.

The `/weather/current/:city` endpoint retrieves current weather details such as temperature, humidity, and condition.

The `/weather/forecast/:city` endpoint provides forecast data

Axios is used to make HTTP requests to the external API, and structured JSON responses are returned to the user.

Task 5:Delete Item

Scenario:

Instructions:

- Use AI to create a DELETE endpoint to remove an item by index or ID.

Starter Code:

```
# DELETE /items/<int:index>

@app.route('/items/<int:index>', methods=['DELETE'])
def delete_item(index):
    if index < 0 or index >= len(items):
        return jsonify({"error": "Item not found"}), 404
    removed_item = items.pop(index)
    return jsonify({"message": "Item deleted", "item": removed_item})
```

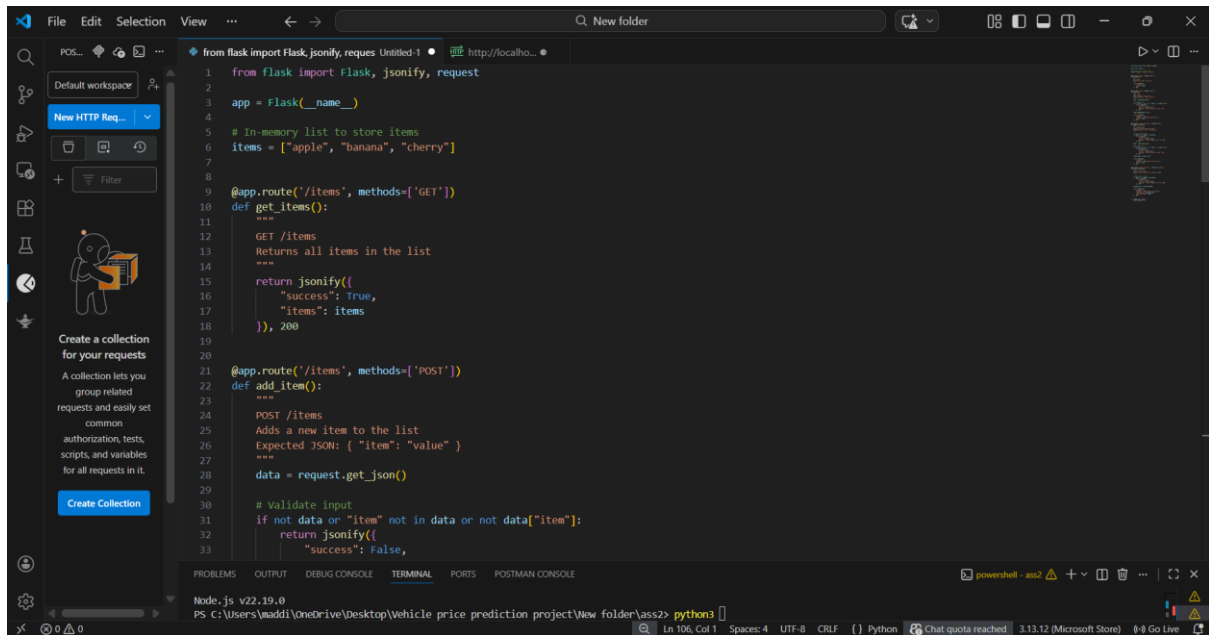
Prompt:

Create a Flask API endpoint to delete an item from a list by index.

Requirements:

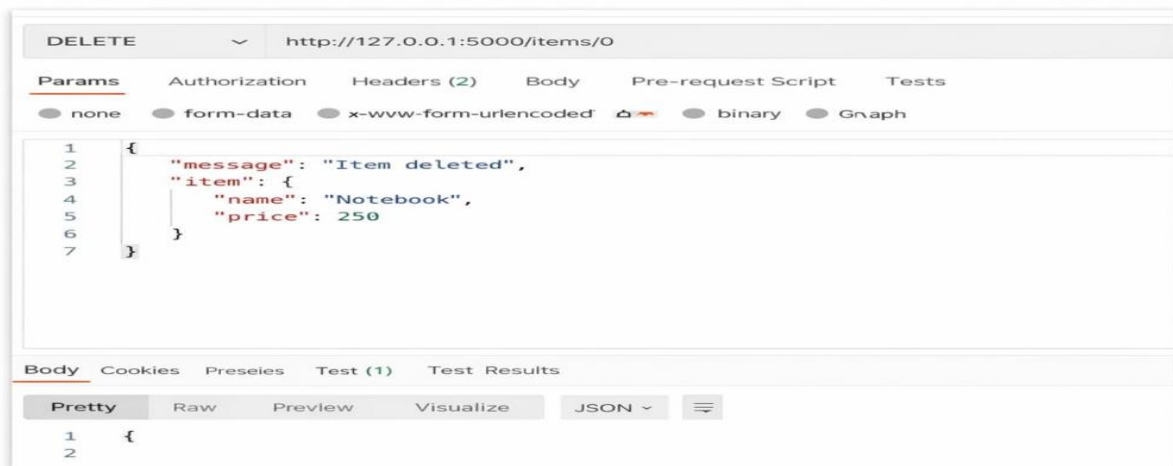
- DELETE /items/<int:index>
- Return error if index invalid
- Return deleted item in response Add proper comments.

Code:



```
1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 # In-memory list to store items
6 items = ["apple", "banana", "cherry"]
7
8
9 @app.route('/items', methods=['GET'])
10 def get_items():
11     """
12     GET /items
13     Returns all items in the list
14     """
15     return jsonify({
16         "success": True,
17         "items": items
18     }), 200
19
20
21 @app.route('/items', methods=['POST'])
22 def add_item():
23     """
24     POST /items
25     Adds a new item to the list
26     Expected JSON: { "item": "value" }
27     """
28     data = request.get_json()
29
30     # Validate input
31     if not data or "item" not in data or not data["item"]:
32         return jsonify({
33             "success": False,
```

Node.js v22.19.0
PS C:\Users\vaadi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> python3



Explanation:

This API demonstrates how to delete an item from a list using a DELETE request. The DELETE /items/<index> endpoint removes an item based on its index.

It includes error handling to check if the index is valid.

If valid, the item is removed and returned in the response.